

Week 10

Binary Search Tree

Outline

This topic covers binary search trees:

- Background
- Definition and examples
- Implementation details of BST operations

Background

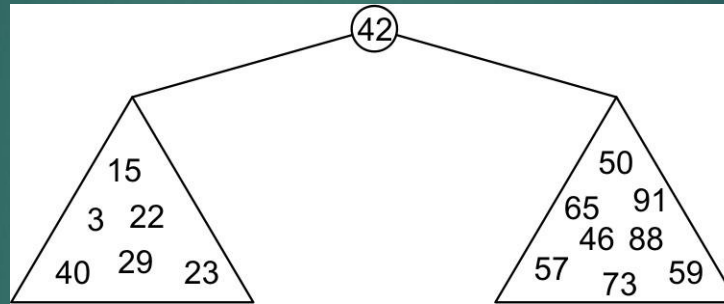
Recall that with a binary tree, we can dictate an order on the two children

We will exploit this order:

- Require all objects in the left sub-tree to be less than the object stored in the root node, and
- Require all objects in the right sub-tree to be greater than the object in the root object

Binary Search Trees

Graphically, we may relationship



- Each of the two sub-trees will themselves be binary search trees

Binary Search Trees

Notice that we have already used this structure for searching: examine the root node and if we have not found what we are looking for:

- If the object is less than what is stored in the root node, continue searching in the left sub-tree
- Otherwise, continue searching the right sub-tree

With a linear order, one of the following three must be true:

$$a < b \quad a = b \quad a > b$$

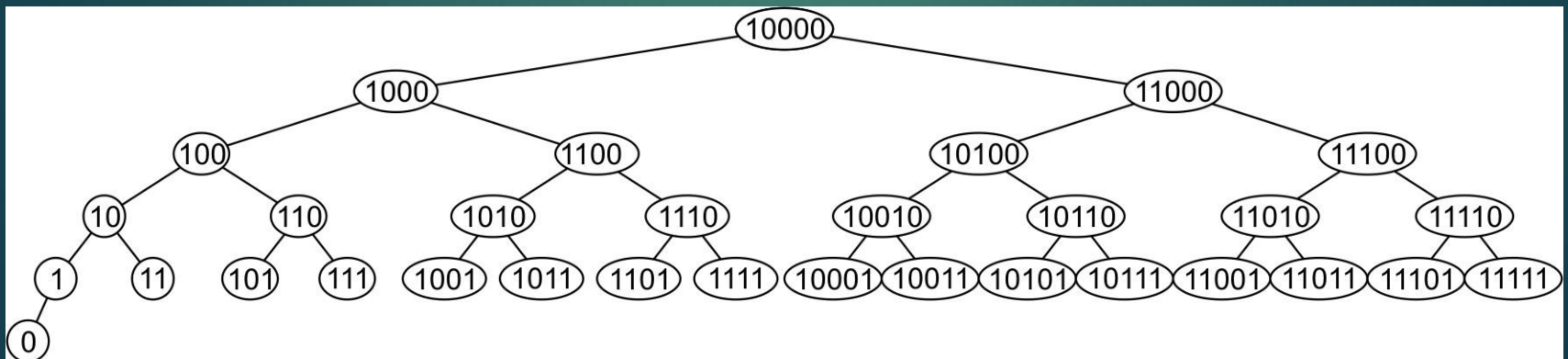
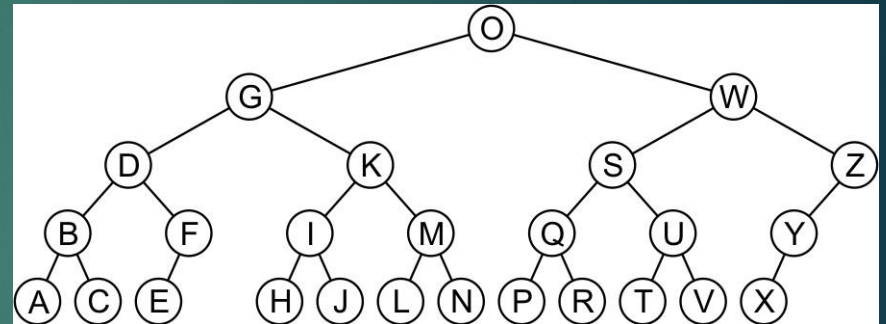
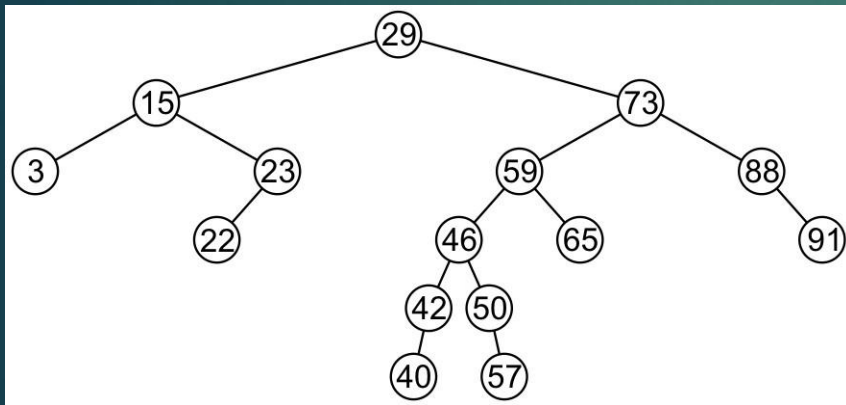
Definition

Thus, we define a non-empty binary search tree as a binary tree with the following properties:

- The left sub-tree (if any) is a binary search tree and all values are less than the root value, and
- The right sub-tree (if any) is a binary search tree and all values are greater than the root value

Examples

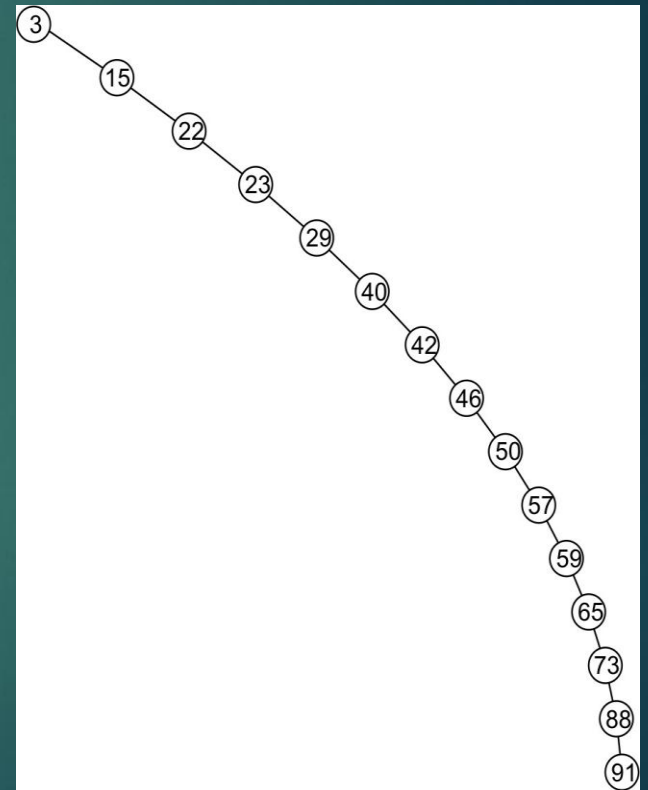
Here are other examples of binary search trees:



Examples

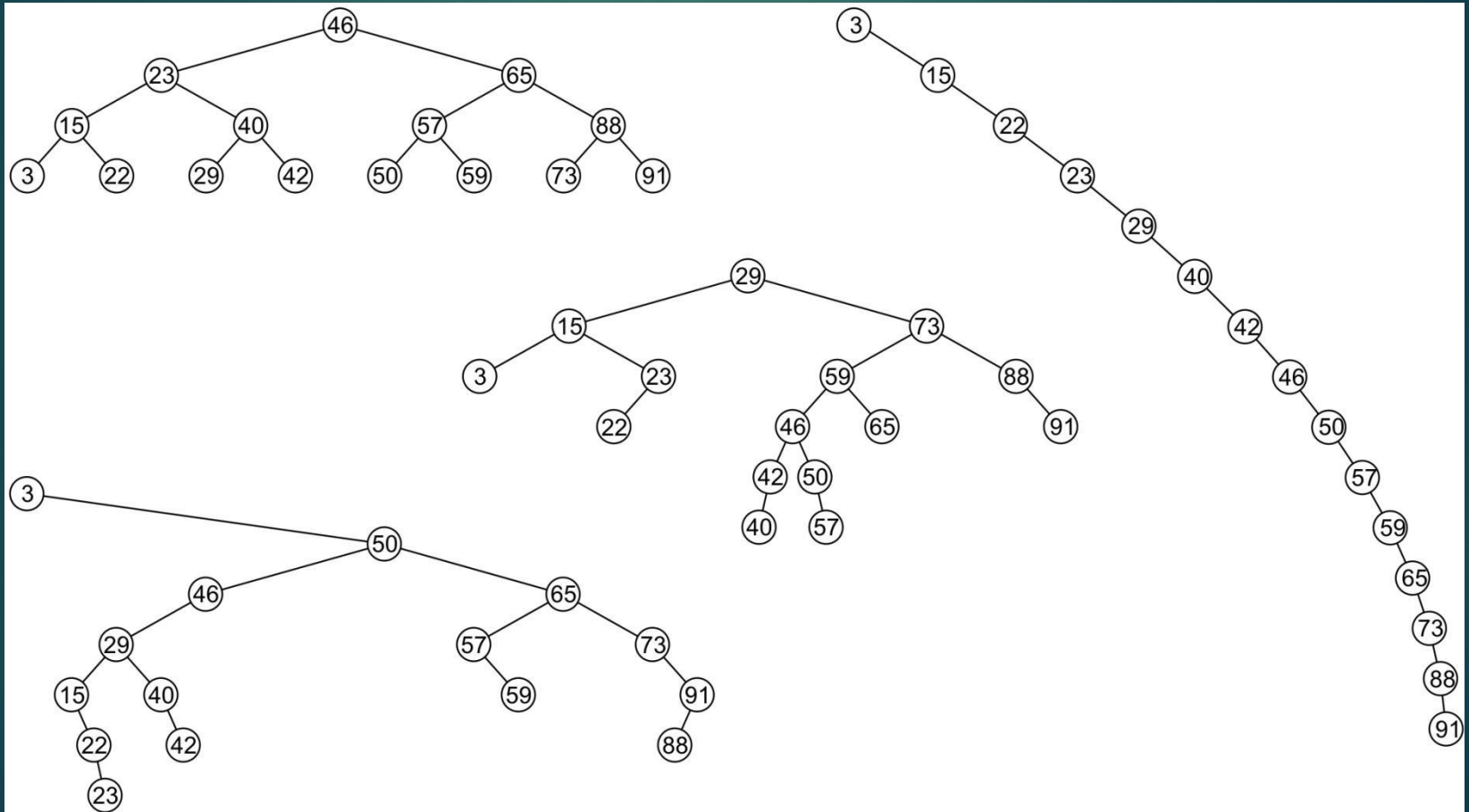
Unfortunately, it is possible to construct *degenerate* binary search trees

- This is equivalent to a linked list, *i.e.*, $O(n)$



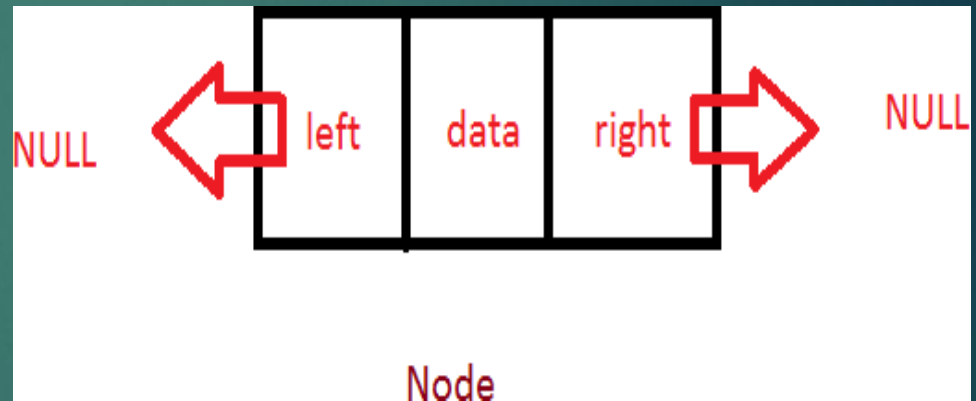
Examples

All these binary search trees store the same data



Structure of BST Node

```
Struct node{  
    int data;  
    node * left;  
    node *right;  
};
```



Operations on a BST

- Insert
- Update
- Search
- Delete
- Traversal
 - BFT
 - DFS
 - In-order, post-order, and pre-order
- Finding minimum
- Finding maximum
- Finding predecessor
- Finding Successor
- Finding height

Insert

- Let's insert any random data into a BST
(Discussion)

Implementation Details

```
Node* insert (int data, node *head)
{
    if(head == NULL)
    {
        head= new node;
        head->data= data;
        head->left= NULL;
        head->right= NULL;
    }
    else if (data < head->data)
        head->left= insert(data, head->left);
    else if(data > head->data)
        head->right= insert(data, head->right);
    return head;
}
```

Search

```
Node* search(node* head, int data){  
    if(head == NULL)  
        return NULL;  
    else if( data < head->data)  
        return search(head->left, data);  
    else if (data > head->data)  
        return search(head->right, data);  
    else  
        return head;  
}
```

Finding minimum

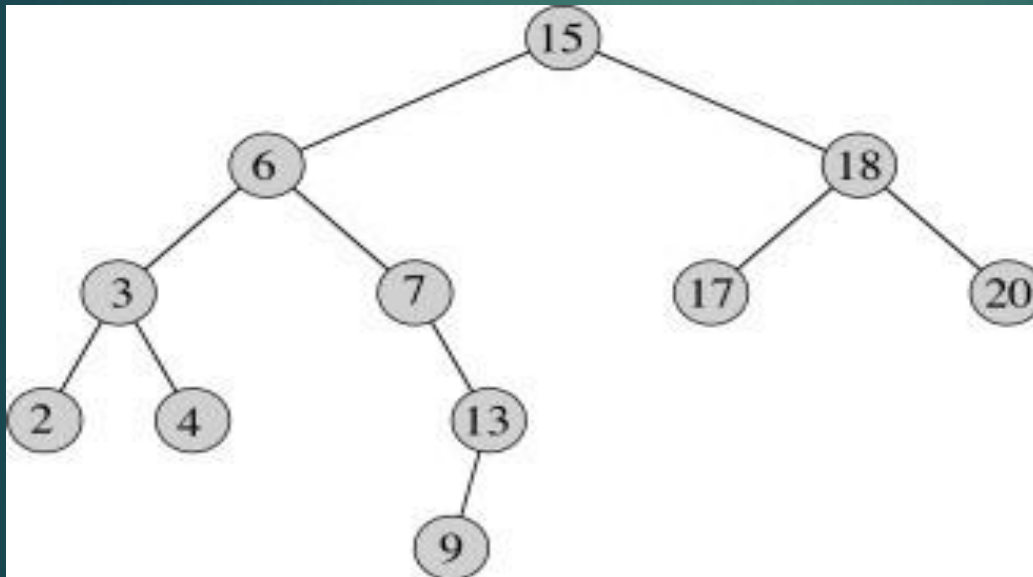
```
Node* min(node *head)
{
    if(head == NULL)
        return NULL;
    else if (head->left == NULL) return head;
    else
        return min(head->left);
}
```

Finding Maximum

```
Node* max(node *head)
{
    if(head == NULL)
        return NULL;
    else if (head->right == NULL)
        return head;
    else
        return max(head->right);
}
```


Finding Predecessor

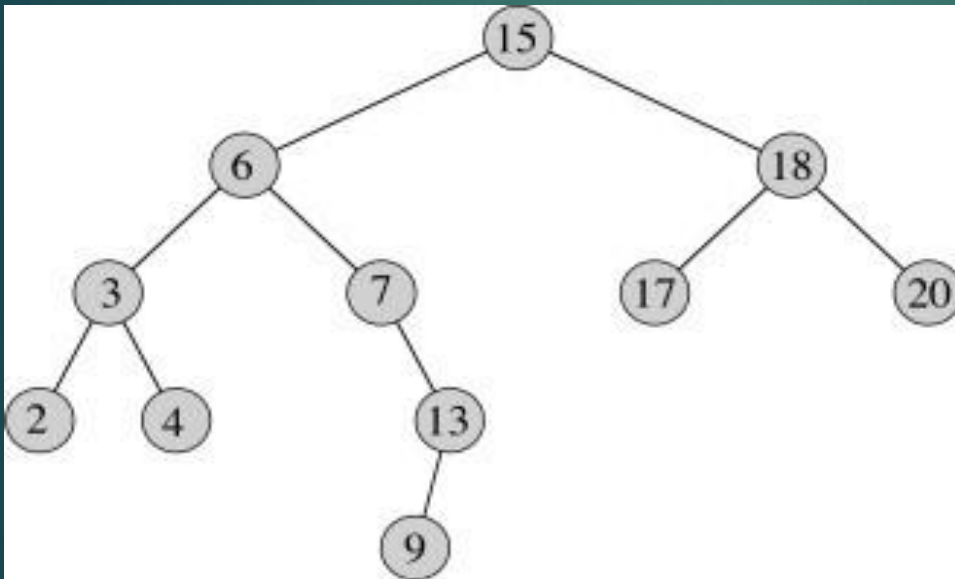
- A predecessor is a maximum value in a left sub tree
- A predecessor of a node is a right most element in a left sub tree.



```
if(head->left!=NULL)  
    return max(head->left)
```

Finding Successor

- A successor is a minimum value in a right sub tree
- A successor of a node is a left most element in a right sub tree.



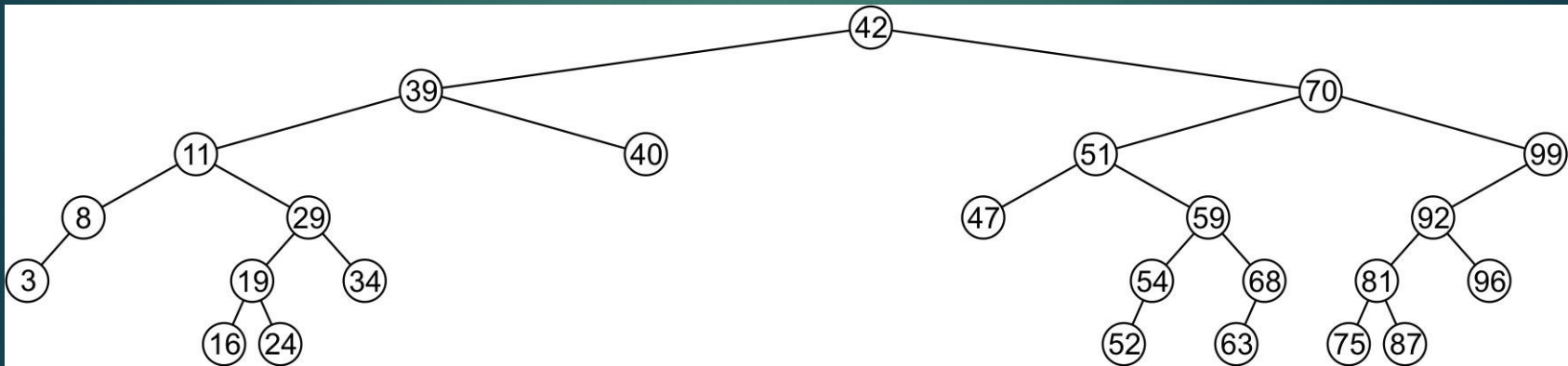
If(head->right!=NULL)
return min(head->right)

Delete

A node being erased is not always going to be a leaf node

There are three possible scenarios:

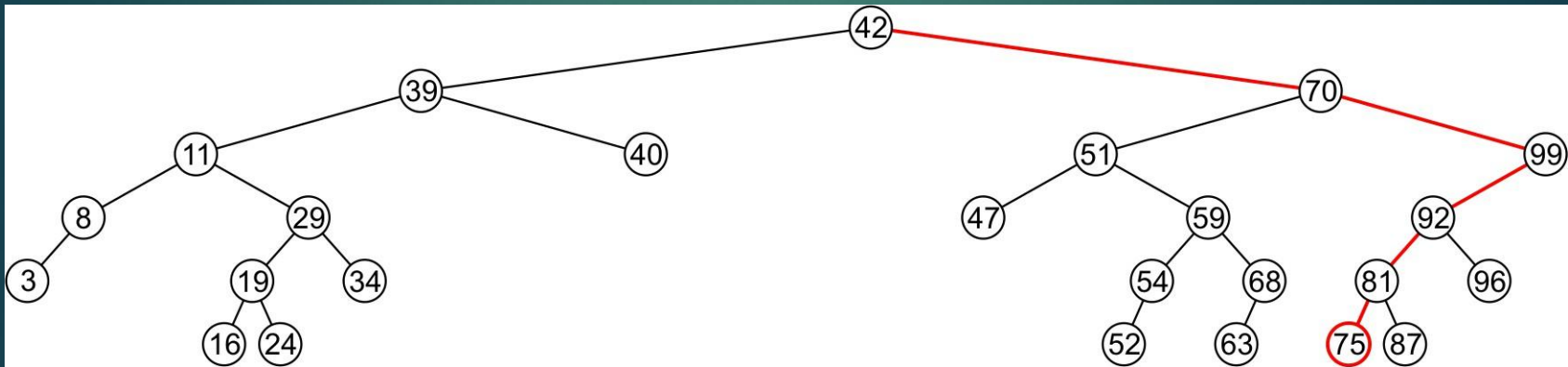
- The node is a leaf node,
- It has exactly one child, or
- It has two children (it is a full node)



Delete

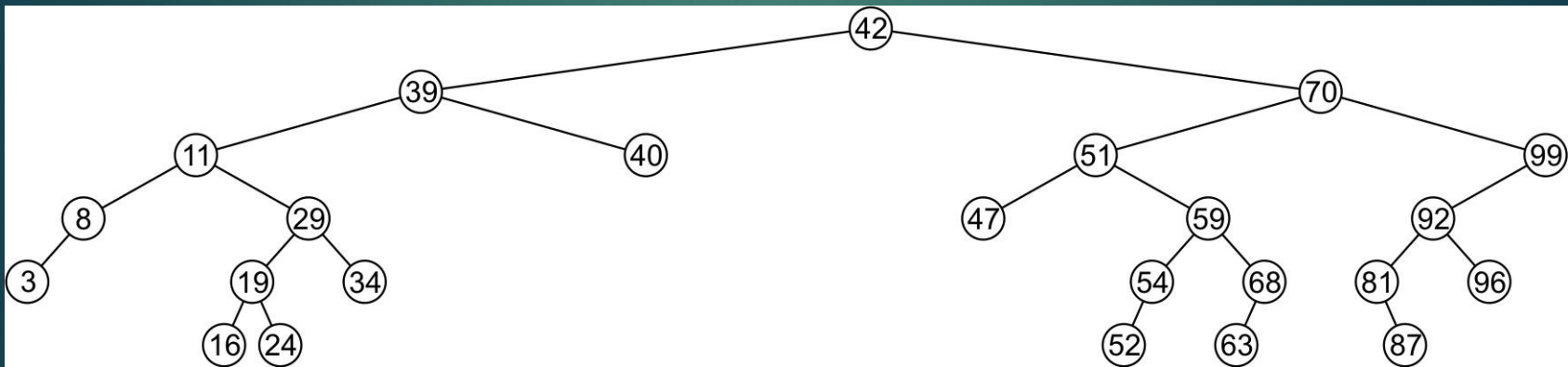
A leaf node simply must be removed and the appropriate member variable of the parent is set to nullptr

- Consider removing 75



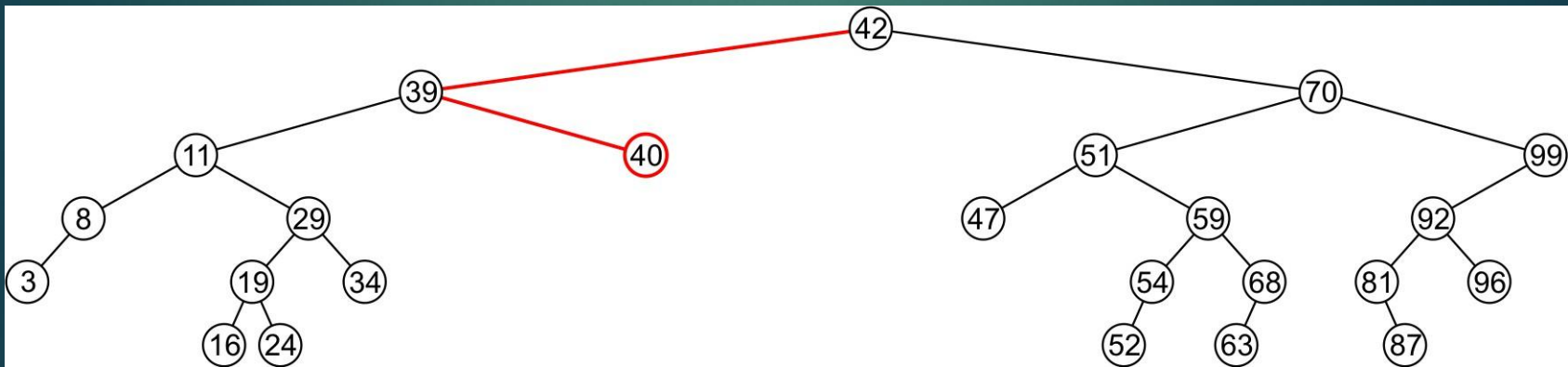
Delete

The node is deleted and left_tree of 81 is set to nullptr



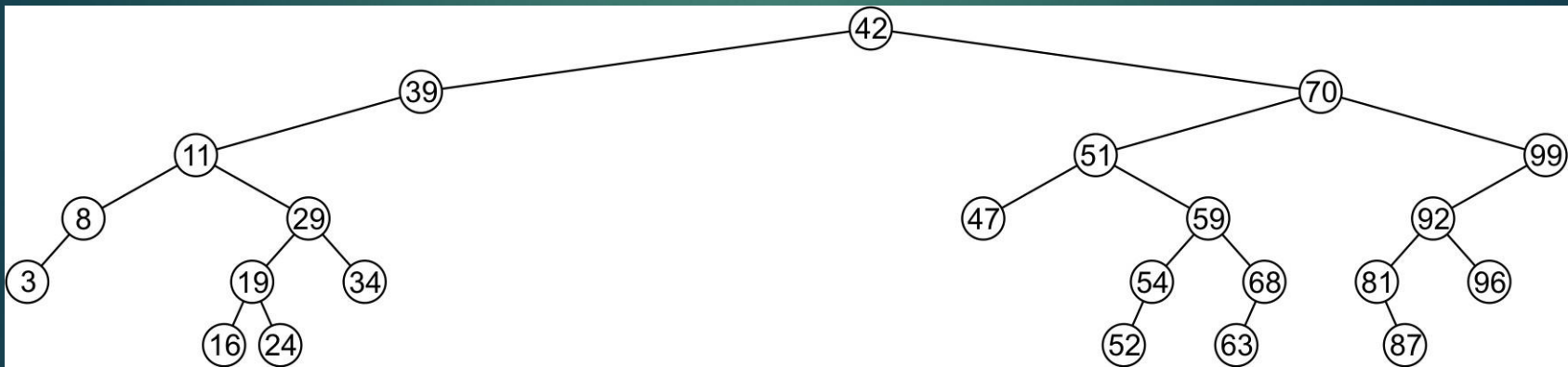
Delete

Erasing the node containing 40 is similar



Delete

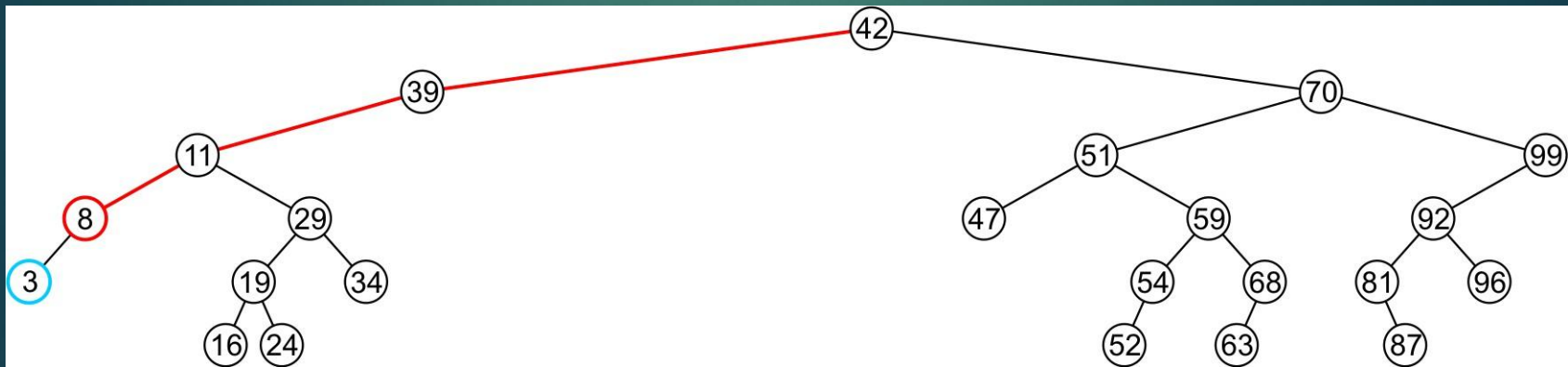
The node is deleted and `right_tree` of 39 is set to `nullptr`



Delete

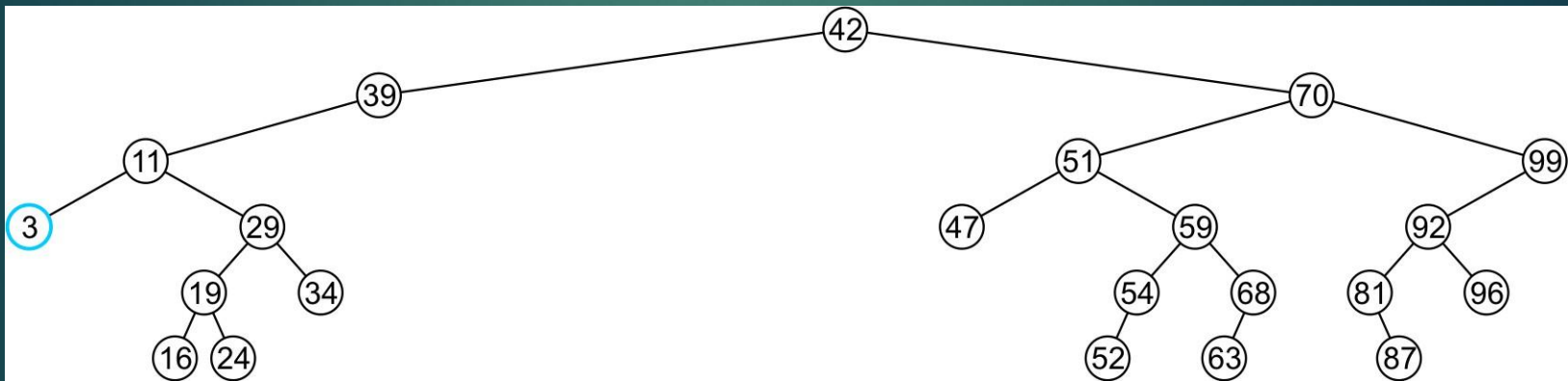
If a node has only one child, we can simply promote the sub-tree associated with the child

- Consider removing 8 which has one left child



Delete

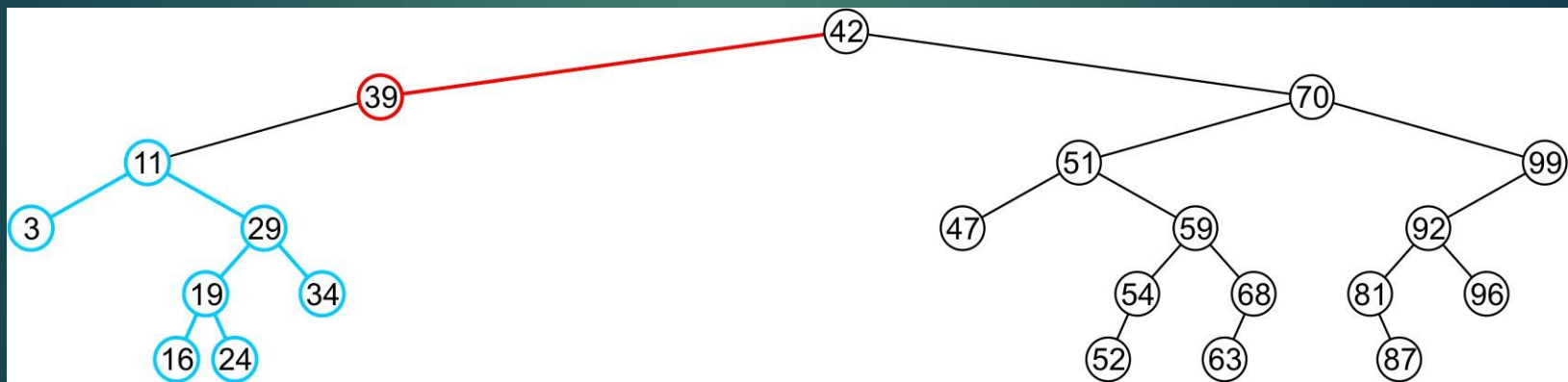
The node 8 is deleted and the left_tree of 11 is updated to point to 3



Delete

There is no difference in promoting a single node or a sub-tree

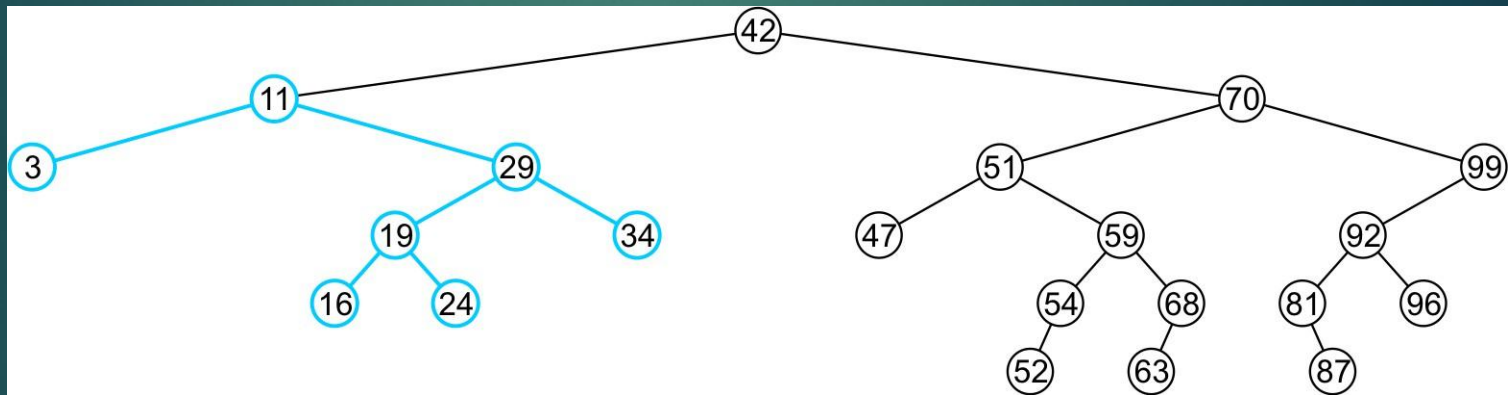
- To remove 39, it has a single child 11



Delete

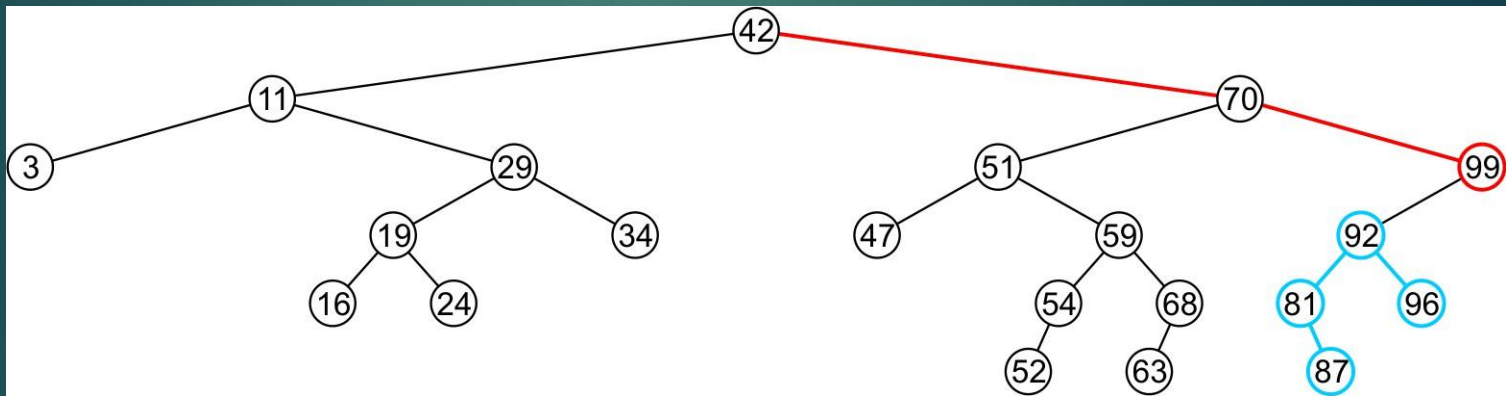
The node containing 39 is deleted and left_node of 42 is updated to point to 11

- Notice that order is still maintained



Delete

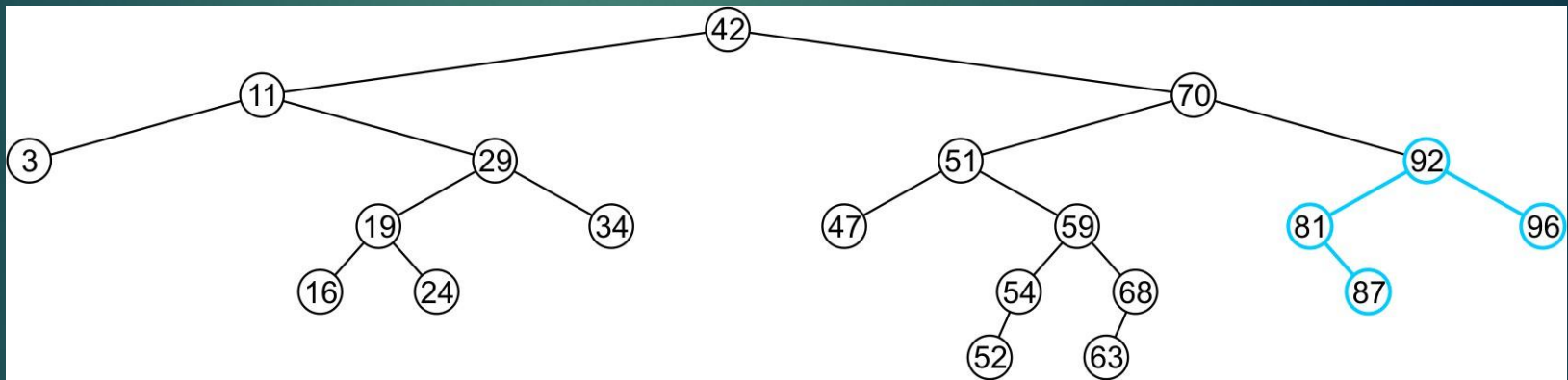
Consider erasing the node containing 99



Delete

The node is deleted and the left sub-tree is promoted:

- The member variable `right_tree` of 70 is set to point to 92
- Again, the order of the tree is maintained

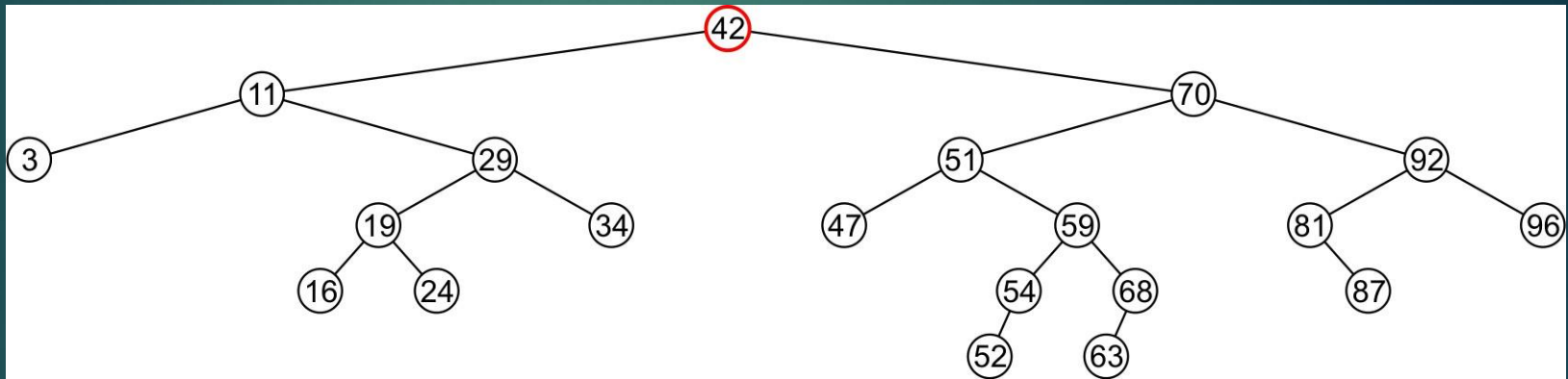


Delete

Finally, we will consider the problem of erasing a full node, e.g., 42

We will perform two operations:

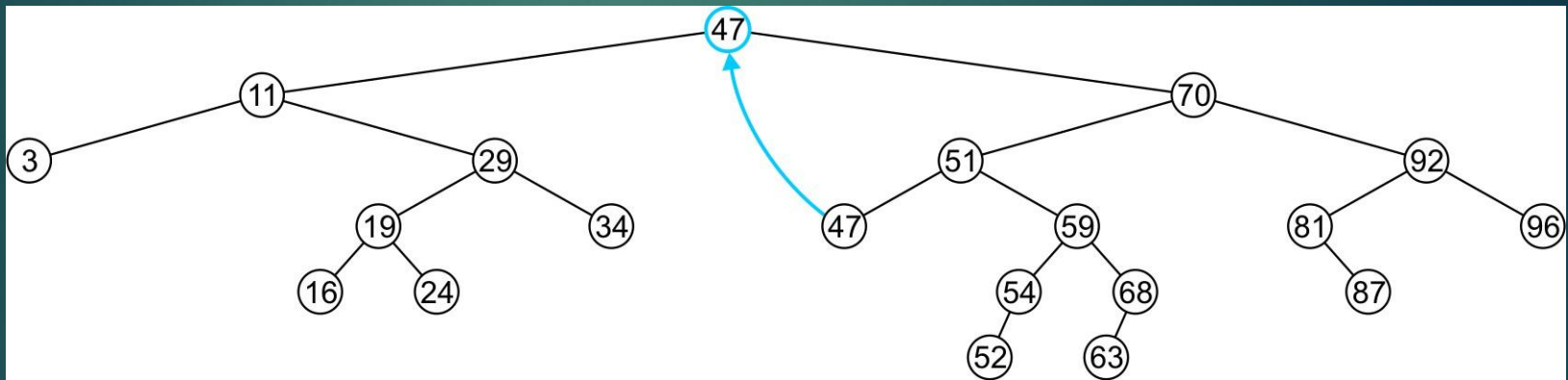
- Replace 42 with the minimum object in the right sub-tree
- Erase that object from the right sub-tree



Delete

In this case, we replace 42 with 47

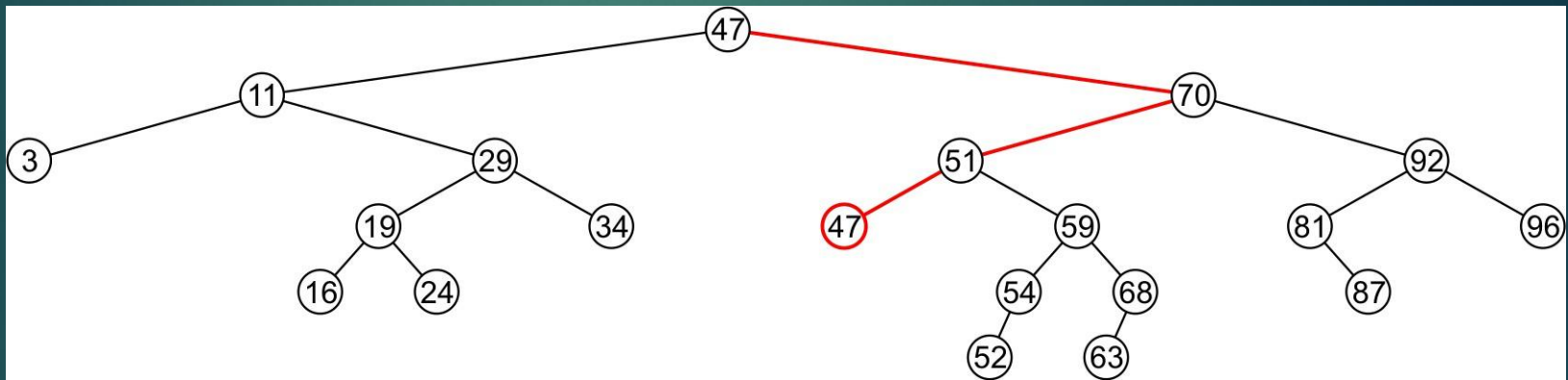
- We temporarily have two copies of 47 in the tree



Delete

We now recursively erase 47 from the right sub-tree

- We note that 47 is a leaf node in the right sub-tree

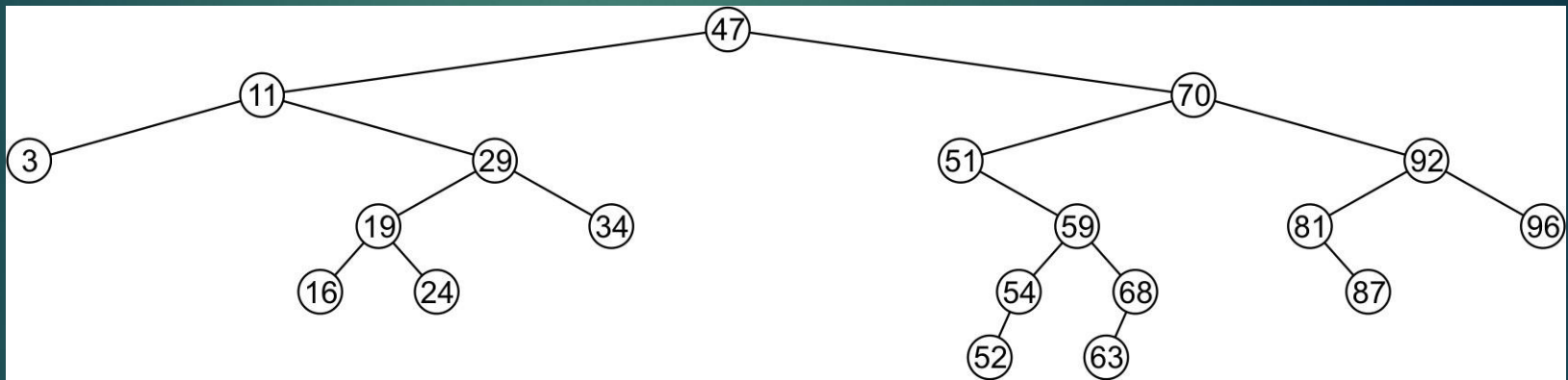


Delete

Leaf nodes are simply removed and `left_tree` of 51 is set to `nullptr`

- Notice that the tree is still sorted:

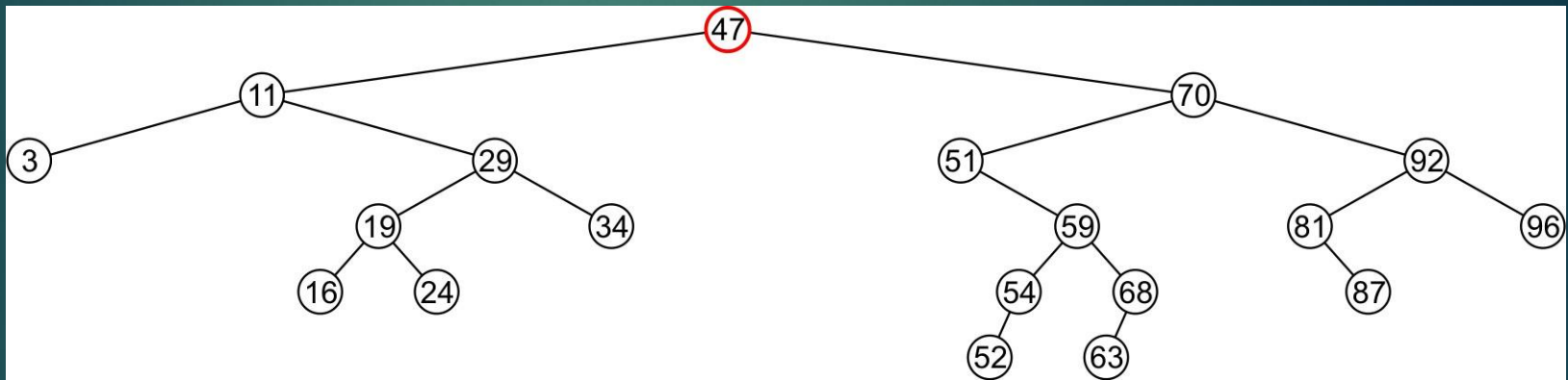
47 was the least object in the right sub-tree



Delete

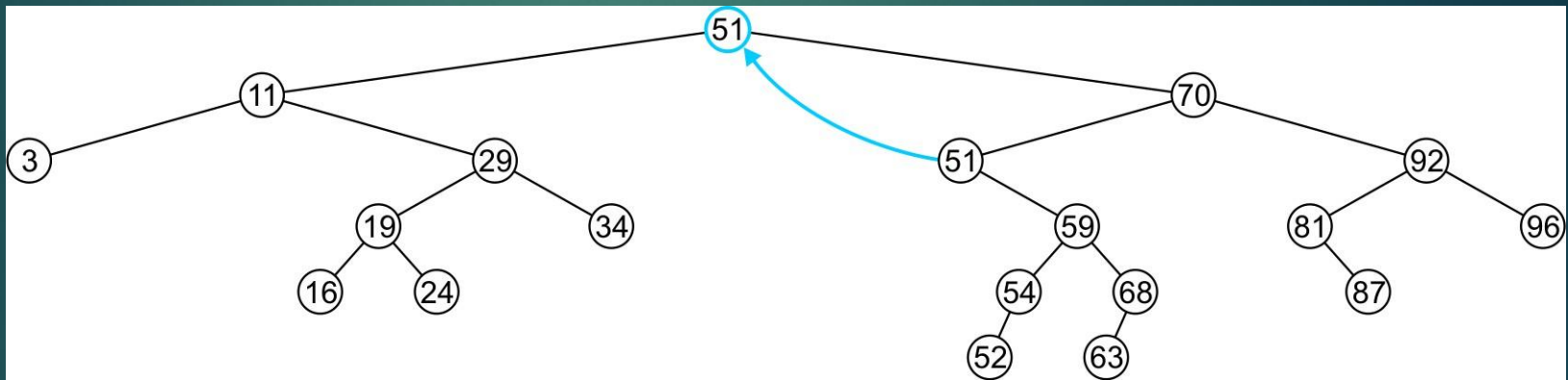
Suppose we want to erase the root 47 again:

- We must copy the minimum of the right sub-tree
- We could promote the maximum object in the left sub-tree and achieve similar results



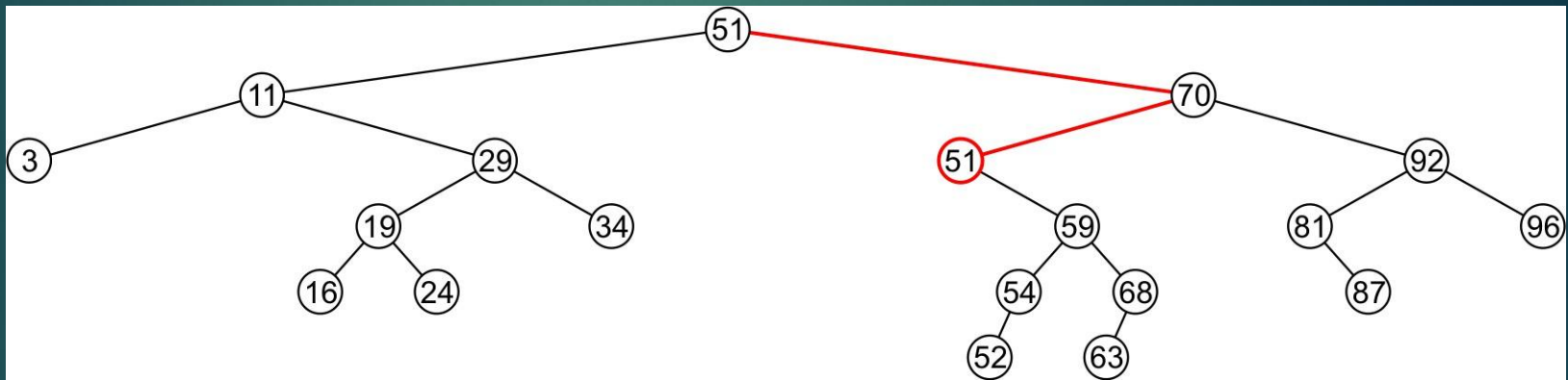
Delete

We copy 51 from the right sub-tree



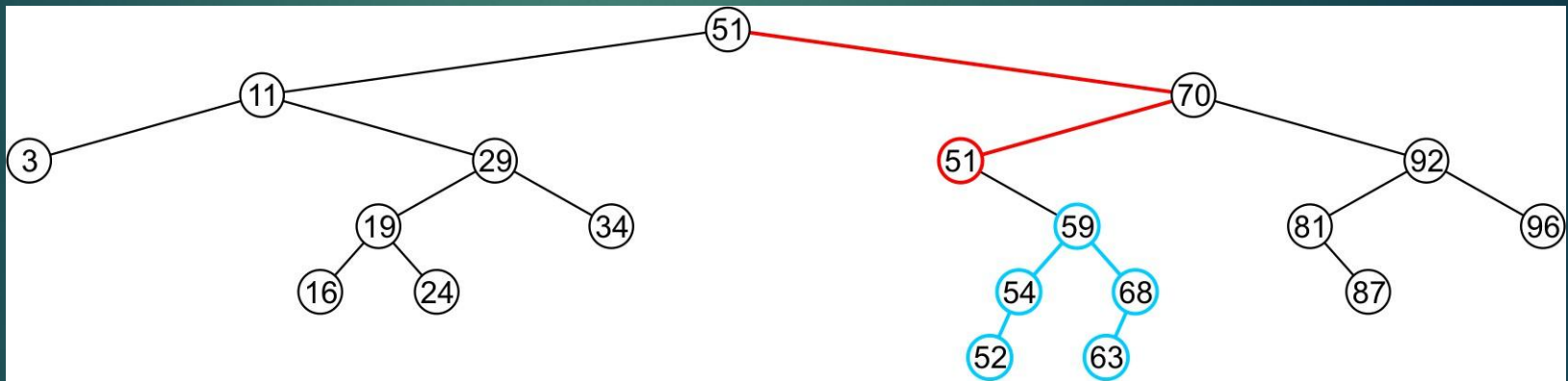
Delete

We must proceed by delete 51 from the right sub-tree



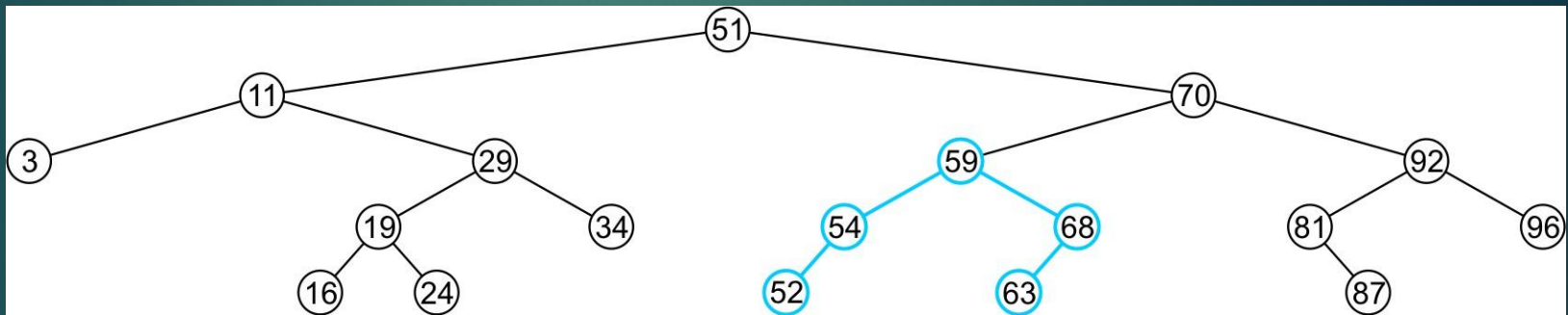
Delete

In this case, the node storing 51 has just a single child



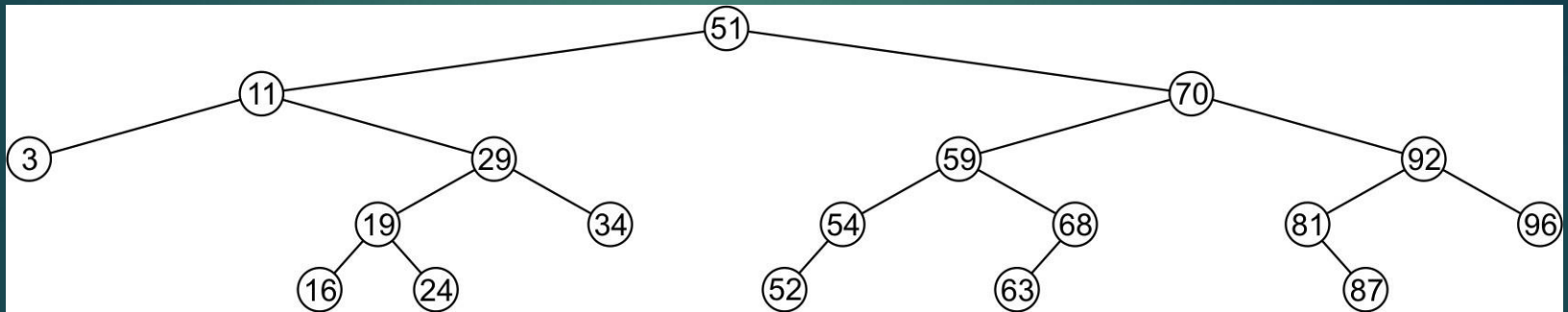
Delete

We delete the node containing 51 and assign the member variable `left_tree` of 70 to point to 59



Delete

Note that after seven removals, the remaining tree is still correctly sorted



Implementation (Delete)

```
Node* deleteNode(Node* root, int key) {
    if (root == NULL)
        return root;
    // Step 1: Search for the node
    if (key < root->data)
        root->left = deleteNode(root->left, key);
    else if (key > root->data)
        root->right = deleteNode(root->right, key);
    else { // Step 2: Node found — handle cases
        // Case 1: No child
        if (root->left == NULL && root->right == NULL) {
            delete root;
            return NULL;
        }
        // Case 2: One child (right)
        else if (root->left == NULL) {
            Node* temp = root->right;
            delete root;
            return temp;
        }
    }
}
```


Implementation (Delete) cont..

```
// Case 2: One child (left)
else if (root->right == NULL) {
    Node* temp = root->left;
    delete root;
    return temp;
}
// Case 3: Two children
else {
    // Find inorder successor (smallest in right subtree)
    Node* temp = root->right;
    while (temp->left != NULL)
        temp = temp->left;
    // Copy successor's data
    root->data = temp->data;
    // Delete the inorder successor
    root->right = deleteNode(root->right, temp->data);
}
}
return root;
}
```

Traversal in a BST

- There are two types of traversals
 - Breadth First Search Traversal (BFS)
 - Depth First Search Traversal (DFS)
 - Make use of Stack ADT
 - Inorder
 - Preorder
 - Postorder

BFS in a BST

- **Breadth-First Search (BFS)** in a **BST** (or any binary tree) means visiting **all nodes level by level, from left to right**.
- We use a **queue** to implement BFS because:
 - The queue follows **FIFO (First In First Out)** — perfect for exploring nodes in the order they appear in levels.
- **Algorithm Steps**
 1. Start with the **root node**.
 2. Push the **root** into a **queue**.
 3. While the queue is **not empty**:
 1. Remove (dequeue) the **front node**.
 2. **Visit** (print) its data.
 3. If it has a **left child**, enqueue it.
 4. If it has a **right child**, enqueue it.

BFS in a BST

- ▶ Let's build this BST:

```
      8
     /\
    3 10
   /\  \
  1 6  14
   /\ /
  4 7 13
```

Step	Queue (front → back)	Node Visited	Output
1	[8]	—	—
2	[3, 10]	8	8
3	[10, 1, 6]	3	8 3
4	[1, 6, 14]	10	8 3 10
5	[6, 14]	1	8 3 10 1
6	[14, 4, 7]	6	8 3 10 1 6
7	[4, 7, 13]	14	8 3 10 1 6 14
8	[7, 13]	4	8 3 10 1 6 14 4
9	[13]	7	8 3 10 1 6 14 4 7
10	[]	13	8 3 10 1 6 14 4 7 13

Final BFS Output:

8, 3, 10, 1, 6, 14, 4, 7, 13

BFS in a BST

```
void BFS(Node* root) {  
    if (root == NULL)  
        return;  
    queue<Node*> q;  
    q.push(root);  
    while (!q.empty()) {  
        Node* current = q.front();  
        q.pop();  
        cout << current->data << " ";  
        if (current->left != NULL)  
            q.push(current->left);  
        if (current->right != NULL)  
            q.push(current->right);  
    }  
}
```

Explanation:

- ✓ Checks if tree is empty
- ✓ Uses a `queue<Node*>` to store nodes
- ✓ Prints each node's data as it's dequeued
- ✓ Enqueues the left and right children (if they exist)

Output Order:

Nodes are printed **level by level**, left to right.

DFS in a BST

- DFS works using a **Stack ADT** (either explicitly with a stack or implicitly using recursion).
- It pushes nodes into the stack as it goes **deeper down the tree**, and pops when it needs to **backtrack**.

► Let's build this BST:

```
      8
     /\
    3 10
   /\  \
  1 6  14
   /\ /
  4 7 13
```

Type	Visiting Order	Output on BST (sorted?)
Inorder	Left → Root → Right	✓ Sorted
Preorder	Root → Left → Right	✗ Not sorted
Postorder	Left → Right → Root	✗ Not sorted

In-order Traversal

► Working Concept (Iterative using Stack)

► In-order means:
Visit Left subtree → Process Root → Visit Right subtree

► To simulate recursion, we use a stack:

- Start from root, keep pushing nodes while going left.
- When no left child, pop the top, print it.
- Move to its right child and repeat.

Final Inorder Output:

1 3 4 6 7 8 10 13 14

Step	Action	Stack (Top → Bottom)	Output
1	Start at 8 → push 8 → move left	[8]	—
2	Push 3 → move left	[3, 8]	—
3	Push 1 → move left	[1, 3, 8]	—
4	No left of 1 → pop → print 1	[3, 8]	1
5	Move right (NULL) → pop next → print 3	[8]	1 3
6	Move right(6) → push 6 → left(4)	[6, 8]	—
7	Push 4 → no left → pop → print 4	[6, 8]	1 3 4
8	Pop 6 → print 6 → go right(7)	[8]	1 3 4 6
9	Push 7 → no left → pop → print 7	[8]	1 3 4 6 7
10	Pop 8 → print 8 → go right(10)	[]	1 3 4 6 7 8
11	Push 10 → no left → pop → print 10 → right(14)	[]	1 3 4 6 7 8 10
12	Push 14 → left(13) → push 13 → pop → print 13	[14]	1 3 4 6 7 8 10 13
13	Pop 14 → print 14	[]	1 3 4 6 7 8 10 13 14

In-order Traversal

```
void inorderDFS(Node* root) {
    stack<Node*> s;
    Node* current = root;

    while (current != NULL || !s.empty()) {
        while (current != NULL) {
            s.push(current);
            current = current->left; // go left as far as possible
        }

        current = s.top();
        s.pop();
        cout << current->data << " ";

        current = current->right; // move to right child
    }
}
```


Pre-order Traversal

- **Working Concept (Iterative using Stack)**
- Visit the **root first**, then **left subtree**, then **right subtree**.
- Stack helps in remembering the path:
 - Push root.
 - Pop → print.
 - Push **right first**, then **left** (so left gets processed first).

Step	Stack (Top → Bottom)	Action	Output
1	[8]	pop 8 → print → push 10,3	8
2	[3,10]	pop 3 → print → push 6,1	8 3
3	[1,6,10]	pop 1 → print	8 3 1
4	[6,10]	pop 6 → print → push 7,4	8 3 1 6
5	[4,7,10]	pop 4 → print	8 3 1 6 4
6	[7,10]	pop 7 → print	8 3 1 6 4 7
7	[10]	pop 10 → print → push 14	8 3 1 6 4 7 10
8	[14]	pop 14 → print → push 13	8 3 1 6 4 7 10 14
9	[13]	pop 13 → print	8 3 1 6 4 7 10 14 13

Final Preorder Output:
8 3 1 6 4 7 10 14 13

Pre-order Traversal

```
void preorderDFS(Node* root) {  
    if (root == NULL)  
        return;  
  
    stack<Node*> s;  
    s.push(root);  
  
    while (!s.empty()) {  
        Node* current = s.top();  
        s.pop();  
        cout << current->data << " ";  
  
        // Push right first so left is processed first  
        if (current->right != NULL)  
            s.push(current->right);  
        if (current->left != NULL)  
            s.push(current->left);  
    }  
}
```

Post-order Traversal

Working Concept (Iterative using Stacks)

• Visit **Left**, then **Right**, and **Root** last.

Algorithm Steps

- Start from the **root**, push it on the stack.
- Go **left** until you reach NULL, pushing nodes.
- Peek the stack's top (don't pop yet):
 - If the node has a **right child not yet visited**, move to right child.
 - Else, **visit (print)** the node and mark it as lastVisited.
- Repeat until stack is empty.

Stack (Top→Bottom)	Current	LastVisited	Output	Action
[8]	3	NULL	—	Go left from 8
[8,3]	1	NULL	—	Go left from 3
[8,3,1]	NULL	NULL	—	Reached NULL, peek=1
[8,3,1]	NULL	NULL	1	1 has no right → visit 1
[8,3]	NULL	1	1	Peek=3 → right=6 not visited → move right
[8,3,6]	4	1	1	Go left from 6
[8,3,6,4]	NULL	1	1	Peek=4 → no right → visit 4
[8,3,6]	NULL	4	1 4	Peek=6 → right=7 not visited → go right
[8,3,6,7]	NULL	4	1 4	Peek=7 → no right → visit 7
[8,3,6]	NULL	7	1 4 7	Peek=6 → right visited → visit 6
[8,3]	NULL	6	1 4 7 6	Peek=3 → right visited → visit 3
[8]	NULL	3	1 4 7 6 3	Peek=8 → right=10 not visited → go right
[8,10]	NULL	3	1 4 7 6 3	Go left (none)
[8,10]	NULL	3	1 4 7 6 3	Peek=10 → right=14 not visited → go right
[8,10,14]	13	3	1 4 7 6 3	Go left
[8,10,14,13]	NULL	3	1 4 7 6 3	Peek=13 → no right → visit 13
[8,10,14]	NULL	13	1 4 7 6 3 13	Peek=14 → right visited → visit 14
[8,10]	NULL	14	1 4 7 6 3 13 14	Peek=10 → right visited → visit 10
[8]	NULL	10	1 4 7 6 3 13 14 10	Peek=8 → right visited → visit 8
[]	NULL	8	1 4 7 6 3 13 14 10 8	Done ✓

Final Postorder Output:

1 4 7 6 3 13 14 10 8

```

void postorder (Node* root) {
    if (root == NULL)
        return;
    stack<Node*> s;
    Node* current = root;
    Node* lastVisited = NULL;
    while (!s.empty() || current != NULL) {
        // Go as left as possible
        if (current != NULL) {
            s.push(current);
            current = current->left;
        }
        else {
            Node* peekNode = s.top();
            // If right child exists and not visited yet → go right

            if (peekNode->right != NULL && lastVisited != peekNode->right)
                current = peekNode->right;
            else {
                cout << peekNode->data << " ";
                lastVisited = peekNode;
                s.pop();
            }
        }
    }
}

```

Finding Height of a BST

```
Int height (node * head)
{
    if(head == NULL)
        return NULL;
    else
    {
        int h_left= height(head->left);
        int h_right = height (head ->right);
        if (h_left > h_right)
            return (h_left + 1); else return (
            h_right + 1);
    }
}
```

Summary

- ▶ In this topic, we covered binary search trees
 - ▶ Described Abstract Sorted Lists
 - ▶ Problems using arrays and linked lists
 - ▶ Definition of a binary search tree
 - ▶ Looked at the implementation of:
 - ▶ Insert
 - ▶ Delete
 - ▶ Search
 - ▶ Traversals
 - ▶ Height